Torneo de Komikan Concurrente — Informe de Diseño

Sistemas Operativos · Tarea 2 · Integrantes: Benjamín Oyarzín · Rodrigo Cantin

1. Diseño del esquema de sincronización

Arquitectura. El servidor es un proceso con hilos: *N* hilos cliente, un hilo *matchmaker*, un hilo cosechador y un hilo de señales. Cada mesa es un **proceso hijo** (el Juez), y el Juez crea dos procesos jugadores: **Puma** (1 hilo) y **Perros** (12 hilos, uno por ficha). El tablero vive en **memoria compartida** (`mmap(MAP_SHARED|MAP_ANONYMOUS)`, creada por el Juez antes del `fork` de los jugadores), de modo que los tres procesos ven el mismo estado. Elegimos procesos (y no solo hilos) porque así una caída de un jugador es real (`SIGKILL`) y el Juez la detecta con `waitpid`, tal como pide la sección 4.1.

```
Servidor (hilos: clientes, matchmaker, cosechador, señales)  -- sem_t mesas_libres = M
  └─ fork -> Mesa/Juez k  (proceso)  -- crea Board en memoria compartida
               ├─ fork -> Jugador Puma   (proceso, 1 hilo)
               └─ fork -> Jugador Perros (proceso, 12 hilos)
```

Primitivas del tablero. (a) **Un mutex por casilla** (`cell_lock[28]`): la casilla es el "slot" que las fichas compiten por reclamar. (b) Un **mutex de estado** (`state_lock`) para lo global: perros vivos, capturas, resultado y el registro circular de eventos. (c) Una **variable de condición** (`state_cond`) con la que el Juez duerme hasta que ocurre un evento. Todos son `PTHREAD_PROCESS_SHARED` (se usan entre procesos) y `PTHREAD_MUTEX_ROBUST`.

Exclusión mutua en movimientos simultáneos. Cada ficha sigue un esquema *optimista + validación pesimista*: (1) duerme 50–300 ms; (2) copia el tablero (`board_peek`) y decide su jugada sobre esa copia; (3) toma los locks de las casillas involucradas (2 para un paso, 3 para una captura: origen, perro y destino) **en orden ascendente de índice**; (4) **revalida** con los locks tomados que la ficha sigue en el origen, que el destino está vacío y, en una captura, que el perro sigue ahí; (5) toma `state_lock`, verifica que la partida siga en curso, escribe el movimiento, registra el evento, aplica las reglas de victoria y avisa al Juez (`broadcast`). Si dos perros apuntan a la misma casilla, ambos piden su lock; el primero en obtenerlo la ocupa y el segundo, al revalidar, la ve ocupada y recibe `MOVE_TAKEN`: perdió la carrera y reintenta en su próximo ciclo. Nunca hay dos fichas en la misma casilla.

Regla de acceso. Para *escribir* `occupant[i]` hay que tener `cell_lock[i]` y `state_lock`; para *leer* basta cualquiera de los dos. Así, una ficha valida con los locks de casilla y el Juez obtiene una **foto consistente** de todo el tablero tomando solo `state_lock`. La victoria (6 capturas o Puma acorralado) se evalúa **dentro de la misma sección crítica** de la jugada: no existe un instante en que el Puma esté acorralado sin que el sistema lo registre.

Servidor y resiliencia. El semáforo POSIX `mesas_libres` (valor inicial *M*) limita las mesas activas: el matchmaker hace `sem_wait` antes de cada `fork`, y el cosechador hace `sem_post` al cosechar una mesa con `waitpid`; los pares sobrantes quedan bloqueados (espera pasiva). Las colas de clientes se protegen con un mutex y tres variables de condición. Si un jugador muere a mitad de partida, el Juez lo detecta con `waitpid(WNOHANG)` y declara ganador por abandono al oponente. Como los mutex son robustos, el siguiente que pida un lock del proceso muerto recibe `EOWNERDEAD`, lo recupera con `pthread_mutex_consistent` y no queda bloqueado. Al cerrar, `board_destroy_shared` hace `trylock` a cada mutex, desbloquea los que quedaron tomados por el muerto, y los destruye. Ante **Ctrl+C**, un hilo dedicado (`sigwait`; SIGINT bloqueada en todos los demás hilos e ignorada en mesas y jugadores) marca el cierre, despierta a todos, envía `SIGTERM` a cada mesa, que aborta su partida, espera a sus jugadores y libera todo; luego el servidor cosecha las mesas y destruye semáforo, mutex y condiciones. La memoria compartida es anónima, así que no deja bloques huérfanos en `/dev/shm`.

2. Análisis de riesgos de concurrencia

Riesgo	Escenario en nuestro diseño	Estrategia de prevención
Deadlock	Perro A en 8 quiere ir a 9 mientras el perro B en 9 quiere ir a 8: A toma 8 y espera 9, B toma 9 y espera 8. También una captura (3 locks) cruzada con pasos de perros.	**Orden global de bloqueo**: casillas siempre en orden ascendente y `state_lock` siempre al final; nunca se pide un lock de casilla teniendo `state_lock`. Con un orden total no puede formarse una espera circular (se rompe una de las condiciones de Coffman). En el ejemplo, A y B piden primero el 8: uno espera al otro y no hay ciclo.

Dueño muerto	Un jugador muere con un lock tomado; con mutex normales todo el que lo pida quedaría bloqueado para siempre (deadlock por recurso huérfano).	Mutex `ROBUST`: `EOWNERDEAD` → `pthread_mutex_consistent`. La jugada se descarta (`MOVE_ERROR`), las fichas vivas siguen esperando y el Juez cierra la partida por abandono. Lo probamos inyectando `SIGKILL` con un lock tomado.
Livelock	Dos perros que se ceden y reintentan la misma jugada en sincronía para siempre, o fichas que se mueven sin que la partida avance nunca.	No hay reintentos inmediatos: tras perder una carrera, la ficha duerme un retardo aleatorio (50–300 ms) y vuelve a decidir con el tablero nuevo, lo que desincroniza a los competidores. Los perros evitan casillas expuestas y cierran el cerco, y un límite de seguridad `max_moves` aborta partidas que no convergen.
Starvation	Una ficha que pierde siempre la carrera por una casilla; o un par de clientes que nunca obtiene mesa.	Las secciones críticas duran microsegundos frente a esperas de 50–300 ms, así que la probabilidad de perder siempre cae geométricamente. Los clientes se atienden en colas **FIFO** y los pares esperan en un semáforo; las mesas terminan (victoria, abandono o límite), así que todo par en espera termina obteniendo cupo.
Fork en multihilo	El servidor hace `fork` teniendo varios hilos: el hijo solo hereda el hilo que llamó, y un lock tomado por otro hilo quedaría tomado para siempre en la copia.	Solo el hilo matchmaker hace `fork`, y el hijo no usa ningún lock del servidor. El log usa `write()` sobre un buffer (no `printf`/stdio), una sola llamada por bloque, con `O_APPEND` para que las líneas de varias mesas no se mezclen.

3. Evaluación

Granularidad. Con un mutex único para el tablero (la alternativa más simple, que tuvimos en una versión inicial) la exclusión es trivial y no hay deadlock posible, pero toda jugada de las 13 fichas queda serializada. Con un mutex por casilla, las fichas en zonas distintas validan y reclaman en paralelo, y la contención aparece solo donde la regla del juego la exige: en la casilla disputada. El costo es mayor complejidad (orden de bloqueo, revalidación, regla de acceso de dos locks), que justificamos porque refleja la consigna de "reclamar un slot" y porque permite razonar formalmente la ausencia de deadlock. `state_lock` sigue siendo un punto común, pero solo protege una escritura de O(1) y el registro del evento; un log necesita un orden total de eventos de todos modos.

Procesos frente a hilos. Los procesos aíslan fallas: un jugador que muere no corrompe la memoria privada del Juez ni del servidor, y el sistema operativo nos entrega la caída (`waitpid`). Pagamos el costo de `fork`, de primitivas compartidas entre procesos y de tener que limpiar con cuidado, a cambio de cumplir la resiliencia de forma real y no simulada.

Rendimiento medido. Con 200 clientes y $M = 8$ (retardos de 0–2 ms), el torneo completa 100 partidas en ≈1,7 s y nunca supera 8 mesas activas. En 300 partidas de estrés y 50 multiproceso, validando invariantes en cada foto (un solo Puma, cada perro vivo exactamente una vez, capturas = perros muertos), obtuvimos 0 fotos inválidas y entre 100 y 1.300 carreras perdidas resueltas correctamente. ThreadSanitizer no detecta data races, y valgrind sobre el servidor tras Ctrl+C reporta 0 bytes en uso y solo los 3 descriptores estándar. Con retardos reales (50–300 ms), una partida dura entre 1 y 5 s: la latencia de una jugada está dominada por el retardo del enunciado, no por la sincronización.

Entrada robusta. El lector de `config.txt` rechaza (con mensaje y código de salida 1) valores vacíos, líneas sin clave o sin `=`, números con basura, negativos o desbordados, rangos invertidos, posiciones fuera del tablero o repetidas y listas de perros incompletas; `tests/test_config.sh` verifica 31 casos borde.

Reproducibilidad. La semilla del config fija las decisiones de cada ficha y los tiempos de llegada (`rand_r` con una semilla derivada por ficha y por cliente). El orden exacto en que se intercalan los hilos depende del planificador del sistema operativo, por lo que dos ejecuciones con la misma semilla pueden diferir: es una propiedad inherente a la concurrencia real y no un error.